



Informe Proyecto 1

Departamento de Ingeniería de Sistemas y Computación
ISIS-1226 Diseño y Programación Orientada a Objetos

Juan David Vasquez Leyva - 202426062

David Hortua - 202510382

Juan Felipe Leon -

Universidad de los Andes

1. El problema que se resuelve

El proyecto tiene como propósito resolver el problema de El Board Game Cafe “Dulces n Dados” el cual es un establecimiento que combina una cafetería que, a su vez, funciona como un lugar para poder jugar juegos de mesa. El negocio requiere un sistema de información que permita gestionar de manera eficiente sus operaciones, abarcando tres grandes áreas: la gestión del inventario de juegos de mesa (préstamo y venta), el funcionamiento del café (empleados, menú y facturas) y las tareas administrativas. Este documento presenta el modelo, los requerimientos funcionales y las restricciones del sistema.

2. Análisis

A partir del análisis del caso de estudio se llegó a los siguientes elementos. El sistema de “Dulces n Dados” se divide en tres grandes categorías: el servicio de préstamo de juegos y reserva de mesas, el servicio de cafetería y restaurante, y el rol del administrador del local. Además, el sistema también cuenta con tres tipos de usuario: Cliente, Empleado (Cocinero o Mesero) y Administrador, cada uno con su login y contraseña.

Con respecto al servicio de préstamo de juegos, para hacer un préstamo de un juego el cliente debe tener una mesa reservada, en la que se especifique el número de personas y si hay menores de edad, verificando que no pase la capacidad máxima del negocio. Existen tres restricciones adicionales según el tipo de juego: los juegos marcados como difíciles necesitan que haya un mesero capacitado para enseñar las reglas. Además, el café maneja un inventario de juegos a la venta, los cuales son distintos a los que están para préstamo, y tanto los clientes como los empleados pueden tenerlos, estos últimos tienen un código de descuento propio.

En cuanto a la cafetería, el menú incluye productos de pastelería y bebida, que pueden ser alcohólicas o calientes. La venta de algunos productos no está permitida dependiendo de los integrantes de la mesa. Por ejemplo, si en la mesa hay niños, entonces no se puede entregar bebidas alcohólicas. También, las bebidas calientes no pueden servirse cuando se está jugando un juego de acción. Todas las ventas tienen impuestos y además le dan punto de fidelidad al cliente y esto puede usarse como descuentos en compras.

Es importante mencionar que los empleados tienen turnos específicos que se repiten cada semana. El sistema debe garantizar que siempre haya al menos 1 cocinero y 2 meseros al mismo tiempo.

Por último, el administrador tiene acceso a la información de todos los juegos que hay en los inventarios de préstamo y de venta. Puede hacer pedidos de juegos nuevos para ambos

inventarios, o reparar juegos. También, tiene un historial completo de todos los préstamos del local y su información. Además, tiene que hacer un informe con todos los datos de los juegos y café, incluyendo fechas, horas, montos, etc.

3. Modelo

El modelo se construyó identificando las entidades principales, sus atributos y las relaciones entre ellas. A continuación se presenta una descripción de cada clase y sus características. En esto nos basamos para hacer el UML.

3.1 Usuario

La clase Usuario es abstracta y tiene las siguientes subclases.

Usuario (abstract) : login : String, contraseña : String.

Cliente : idCliente, juegosComprados, juegosFavoritos, juegosReservados (maximo 2), puntosDeFidelidad, mesa.

Administrador : Hereda usuario y no tiene más atributos propios.

Empleado (abstract) : turnoLaboral, codigoDescuento, juegosFavoritos. Está clase va a ser la superclase de Cocinero y Mesero.

Cocinero (hereda Empleado) : Hereda empleado y tiene mínimo un turno.

Mesero (hereda Empleado) : juegosDificiles : List<JuegoDeMesa>. Debe tener mínimo 2 turnos.

3.2 Tienda de Juegos

Gestiona el inventario de juegos para préstamo y venta.

TiendaDeJuegos : facturas : List<Factura>, inventarioJuegos : InventarioJuegos

InventarioJuegos : juegosPrestamo, juegosVenta, historialPrestamos : List<Prestamo>

JuegoDeMesa : idJuego, enUso, añoPublicacion, empresaProduccion, tipoDeJuego, minJugadores, maxJugadores, edadMinima, categoria (Cartas / Tablero / Acción), estado, dificil.

Prestamo : idJuego, idMesa, fechaInicio, fechaFin, cliente.

3.3 Cafe

Gestiona el funcionamiento de la cafetería.

Cafe : mesa : List<Mesa>, capacidadMaxClientes : int, empleados : List<Empleado>, menuProductos : List<Producto>, facturas : List <Factura>

Mesa : idMesa, cantidadClientes, tieneNinos, tieneJovenes, cliente.

Turno : diaSemana : datetime, empleado : Empleado.

Producto (abstract) : nombre, precio. Esta clase es la superclase de Bebida y Pasteleria.

Bebida (hereda Producto) : alcoholica : boolean, caliente : boolean.

Pasteleria (hereda Producto) : alergenos : List<String>

3.4 Factura

Gestiona los registros de las ventas.

Factura : fecha, subtotal, impuestos, propina, detalles : List<DetalleVenta>

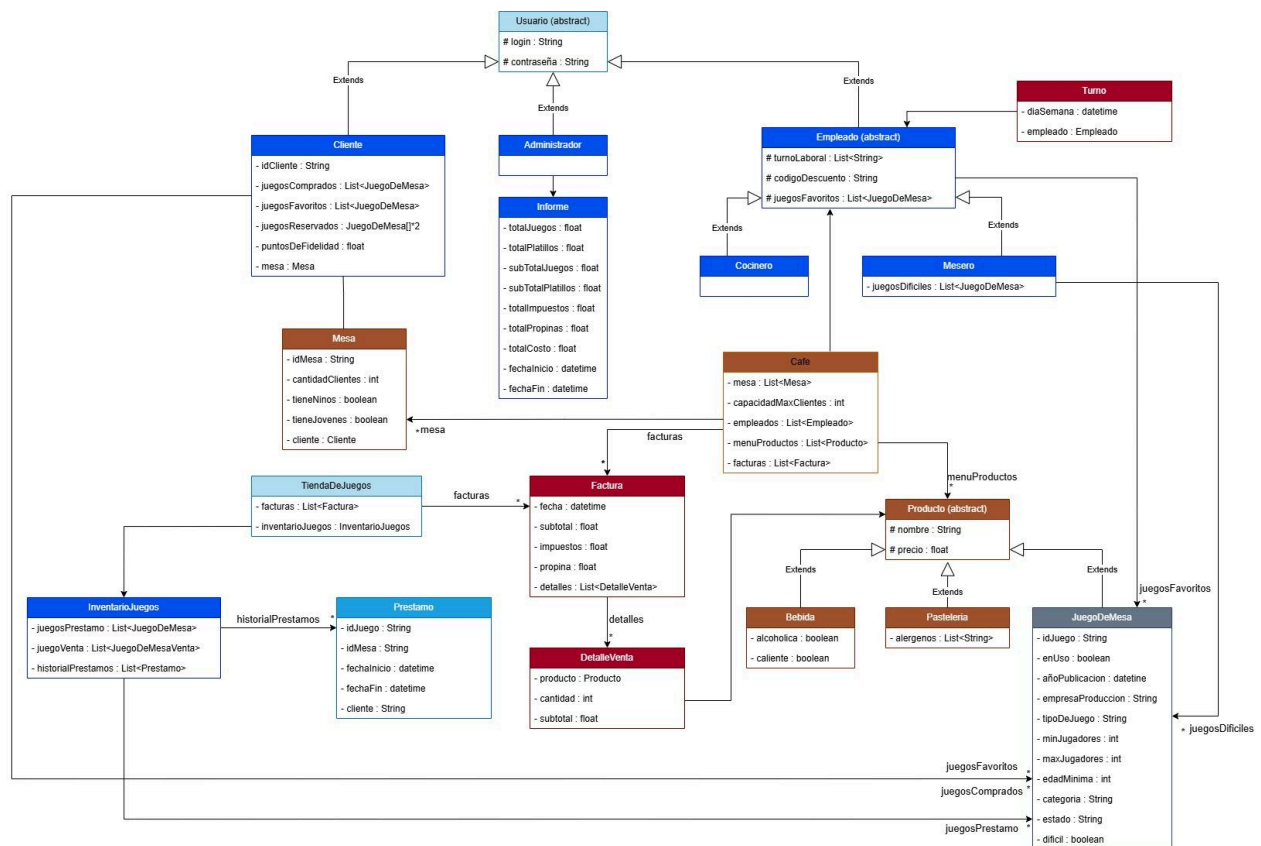
DetalleVenta : producto : Producto, cantidad : int, subtotal : float.

3.5 Admin e Informe

La clase Administrador debe hacer un informe que debe contener lo siguiente.

Informe : totalJuegos, totalPlatillos, subTotalJuegos, subTotalPlatillos, totalImpuestos, totalPropinas, totalCosto, fechaInicio, fechaFin.

4. Diagrama de clases



En este diagrama de clases en UML se ve representado el sistema de “Dulces n Dados”. Se pueden apreciar las clases de Empleado, Administrador, y Cliente, las cuales heredan a Usuario. Esto se debe a que todos los usuarios sin importar su tipo, requieren login y contraseña, y tenerlos como una extensión de Usuario evita la repetición de código.

La clase Empleado es abstracta y tiene como subclases a Mesero y Cocinero. El mesero debe conocer juegos difíciles, mientras que el cocinero es un empleado común y corriente. Por su parte, el Administrador tiene un historial completo de los préstamos realizados en el local, junto con su información, además, debe hacer informes detallados de las ventas.

El cliente se relaciona con la mesa a través de la reserva y con los juegos de mesas por medio de la compra o pedirlos prestados. Además, al tener una mesa, puede hacer uso de las funciones de la cafetería, como ordenar productos tanto de pastelería como bebidas. Es importante mencionar que, tanto las bebidas, los productos de pastelería y los juegos de mesa son herederos de la clase Producto. Cada vez que se hace una compra, al cliente se le genera una factura por cada compra de juegos y cada gasto en el café.

Finalmente, es importante mencionar que los inventarios de préstamo y de venta se separaron dentro de la clase InventarioJuegos para mantener la distinción entre ambos, ya que el texto menciona que ambas tienen listas completamente diferentes.

5. Requerimientos Funcionales

Lista Exceptions:

- *EntidadNoEncontradaException*
- *CapacidadExcedidaException*
- *JuegoNoDisponibleException*
- *RestriccionEdadException*
- *RestriccionJugadoresException*
- *ConflictoHorarioException*
- *OperacionNoPermitidaException*
- *InventarioInsuficienteException*
- *SolicitudNoValidaException*
- *EstadoInvalidoException*

- **reservarJuego**

Entrada:

- clientId: String
- juegoId: String
- mesaId: String

Salida:

- Prestamo

Excepciones:

- JuegoNoDisponibleException
- RestriccionEdadException
- RestriccionJugadoresException
- CapacidadExcedidaException

- **venderJuego**

Entrada:

- clientId: String
- juegoId: String
- cantidad: int

Salida:

- Factura

Excepciones:

- InventarioInsuficienteException
- EntidadNoEncontradaException

- **repararJuego**

Entrada:

- juegoId: String

Salida:

- void

Excepciones:

- EstadoInvalidoException
- EntidadNoEncontradaException

- **darJuegoPorRobado**

Entrada:

- juegoId: String

Salida:

- void

Excepciones:

- EntidadNoEncontradaException

- **comprarJuegoCliente**

Entrada:

- clienteId: String
- listaJuegos: List<JuegoDeMesa>

Salida:

- Factura

Excepciones:

- InventarioInsuficienteException

- **asignarHorario**

Entrada:

- empleadoId: String
- turno: Turno

Salida:

- void

Excepciones:

- ConflictoHorarioException

- **solicitarCambioHorario**

Entrada:

- empleadoId: String

- turnoActual: Turno

- turnoNuevo: Turno

Salida:

- SolicitudCambio

Excepciones:

- SolicitudNoValidaException

- **solicitarIntercambioHorario**

Entrada:

- empleado1Id: String

- empleado2Id: String

- turno1: Turno

- turno2: Turno

Salida:

- SolicitudCambio

Excepciones:

- SolicitudNoValidaException

- **aprobarSolicitudHorario**

Entrada:

- administradorId: String

- solicitudId: String

Salida:

- void

Excepciones:

- EntidadNoEncontradaException

- **sugerirPlatillo**

Entrada:

- empleadId: String
- producto: Producto

Salida:

- void

Excepciones:

- OperacionNoPermitidaException

- **aceptarSugerenciaPlatillo**

Entrada:

- administradorId: String
- producto: Producto

Salida:

- void

Excepciones:

- EntidadNoEncontradaException

- **agregarPlatillo**

Entrada:

- producto: Producto

Salida:

- void

Excepciones:

- EstadoInvalidoException

- **agregarPuntosDeFidelidad**

Entrada:

- clienteId: String
- montoCompra: float

Salida:

- float (puntos agregados)

Excepciones:

- EntidadNoEncontradaException

- **consultarEstadoJuego**

Entrada:

- juegoId: String

Salida:

- String (estado)

Excepciones:

- EntidadNoEncontradaException

- **consultarNumPrestamosJuego**

Entrada:

- juegoId: String

Salida:

- int

Excepciones:

- EntidadNoEncontradaException

- **consultarFechaPrestamoJuego**

Entrada:

- juegoId: String

Salida:

- List<datetime>

Excepciones:

- EntidadNoEncontradaException

- **moverJuegoDelInventarioAVentas**

Entrada:

- juegoId: String

Salida:

- void

Excepciones:

- InventarioInsuficienteException

- **solicitarJuegoPrestado**

Entrada:

- clientId: String
- mesId: String
- juegoId: String

Salida:

- Prestamo

Excepciones:

- JuegoNoDisponibleException
- RestriccionEdadException
- RestriccionJugadoresException

- **reabastecerInventarioVentas**

Entrada:

- juegoId: String
- cantidad: int

Salida:

- void

Excepciones:

- EstadoInvalidoException

- **reabastecerInventarioPrestamo**

Entrada:

- juegoId: String
- cantidad: int

Salida:

- void

Excepciones:

- EstadoInvalidoException

- **reportarJuegoRobado**

Entrada:

- juegoId: String

Salida:

- void

Excepciones:

- EntidadNoEncontradaException

- **asignarTurnoHorario**

Entrada:

- empleadId: String
- turno: Turno

Salida:

- void

Excepciones:

- ConflictoHorarioException

- **modificarTurnoHorario**

Entrada:

- empleadId: String
- turnoAntiguo: Turno
- turnoNuevo: Turno

Salida:

- void

Excepciones:

- ConflictoHorarioException

- **generarInforme**

Entrada:

- fechaInicio: datetime
- fechaFin: datetime

Salida:

- Informe

Excepciones:

- SolicitudNoValidaException